format) needs to be selected after the application has begun to execute, and the selection code might look something like this:

```
// request URL contains query string '?format=$FILE_FORMAT'
$format = $_GET['format'];

switch ($format) {
    case 'csv':
        $csvFormatter->buildOutput($data);
        break;
    case 'psv':
        $psvFormatter->buildOutput($data);
        break;
    case 'tsv':
        $tsvFormatter->buildOutput($data);
        break;
    case 'xls':
        $xlsFormatter->buildOutput($data);
        break;
}
```

Such code is fine as it goes but can become unwieldy as new behaviors (in our case, supported file formats) are introduced. Also, notice that the client code holds references to all the possible output formatting objects, despite only ever needing one of them. It is exactly this scenario that the *Strategy Pattern* is designed for. With this pattern, each algorithm is wrapped in a class that conforms to a common `Strategy` interface. At run time, another object – called a `Context` – is used to wrap the chosen strategy, and the client code is given a reference to the context. For our hypothetical problem the class diagram would look like this: